COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S4
DATE PERFORMED:	24/07/2026	**DATE SUBMITTED:**	24/07/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	Engir. Jim lord Quiejado

ACTIVITY NO. 3.1 **Linke lists**

1. PROCEDURE OUTPUT

What procedures did you perform?

OUTPUT:

Screenshot	
Discussion	

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot

Traversal	main.cpp > main()
	#include <iostream>
	#include "singly_ll.h"
	#include "doubly_ll.h"
	
	int main() {
	// C P E
	DoubleList<char>* head = createNode('C');
	DoubleList<char>* second = createNode('P');
	DoubleList<char>* third = createNode('E');
	
	// connecting the nodes:
	
	// connect the head node to the next node:
	head->next = second;
	second->prev = head;
	
	//
	second ->next = third;
	third ->prev = second;
	
	// traversal
	std::cout << "Testing the dllTraverse List" << std::endl;
	dllTraverse(head);
	
	//insert at the head
	std::cout<<"Testing the dllInsertHead"<<std::endl;
	dllInsertHead('X', &head);
	dllTraverse(head);
	
	//insert at the end
	//insert at the head
	std::cout<< "Testing the dllInsertENd" <<std::endl;
	dllInsertEnd('0', head);
	dllTraverse(head);
	
	//general insert
	std::cout << "\nTesting General Insert" << std::endl;
	dllGeneralInsert('1', head->next->next);
	dllTraverse(head);

Insertion at head	

Insertion at any part of the list	
Insertion at the end	

main.cpp > main()

```cpp
#include <iostream>
#include "singly_ll.h"
#include "doubly_ll.h"

int main() {
    // C P E
    DoubleList<char>* head = createNode('C');
    DoubleList<char>* second = createNode('P');
    DoubleList<char>* third = createNode('E');

    // connecting the nodes:

    // connect the head node to the next node:
    head->next = second;
    second->prev = head;

    //
    second ->next = third;
    third ->prev = second;

    // traversal
    std::cout << "Testing the dllTraverse List" << std::endl;
    dllTraverse(head);

    //insert at the head
     std::cout<<"Testing the dllInsertHead"<<std::endl;
     dllInsertHead('X', &head);
     dllTraverse(head);

     //insert at the end
        //insert at the head
     std::cout<< "Testing the dllInsertENd" <<std::endl;
     dllInsertEnd('0', head);
     dllTraverse(head);

     //general insert
     std::cout << "\nTesting General Insert" << std::endl;
     dllGeneralInsert('1', head->next->next);
     dllTraverse(head);

```

```
    //deleting a node
    std::cout << "\nTesting delete a node" << std::endl;
    dllDeleteNode('X', &head);
    dllTraverse(head);
    dllDeleteNode('0', &head);
    dllTraverse(head);



return 0;

}
```

C singly_ll.h > ...

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SinglyList{
    public:
    T data;
    SinglyList<T>* next;
};

template <typename T>
void ListTraversal(SinglyList<T>* node){

    if(node == nullptr){
        std::cout << "the list is empty" << std::endl;
        return;
    }

    //WHILE n IS NOT EQUAL TO null:
    while(node != nullptr){
        //PRINT data to n
        std::cout << node->data;

        if(node->next != nullptr){
            std::cout << "->";
        }

        //GO TO THE NEXT NODE:
        node = node->next;

    }
    //PRINT THE NEXT LINE
    std::cout << std::endl;
}

template <typename T>
void sllInsertHead(T newData, SinglyList<T>** currentHead){

    // 1. Allocate memeory for then new node
    SinglyList<T>* newNode = new SinglyList<T>;

    // 2. Put out data into the new node
    newNode->data = newData;

```

```cpp
}

template <typename T>
void sllGeneralInsert(T newData, SinglyList<T>* prevNode){

    // 1. Check if it is the head node
    if(prevNode == nullptr){
        // 2. If null, print "Previous node cannot be null."
        std::cout << "Previous node cannot be null." << std::endl;
    }
    // 3. Allocate a new node
    SinglyList<T>* newNode = new SinglyList<T>;
    // 4. Store data in the new node
    newNode->data = newData;
    // 5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // 6. Point previous node to the new node
    prevNode->next = newNode;
}

template <typename T>
void sllInsertEnd(T newData, SinglyList<T>** head){
    // 1. Allocate new node
    SinglyList<T>* newNode = new SinglyList<T>;
    //dereference to the new node
    SinglyList<T>* currentNode = *head;
    //store data in new node:
    newNode->data = newData;
    //point the new node to null
    newNode->next = nullptr;

    //Traverse the list of the current node to the new node
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //Point the next of the current node to the new node
    currentNode->next = newNode;

}
```

```
92  template <typename T>
93  void sllDeleteNode(T findData, SinglyList<T>** head){
94      //check first if the list is already empty??
95      if(*head == nullptr) return;
96
97      SinglyList<T>* currNode= *head;
98      SinglyList<T>* prevNode= nullptr;
99
100     //search for the node 1. what if the data we are looking for is not... (idk)
101     // 2. the data is found
102     while(currNode != nullptr && currNode->data != findData) {
103         prevNode = currNode;
104         currNode = currNode->next;
105     }
106
107     //data was not found
108     if(currNode == nullptr) return;
109
110     if(prevNode == nullptr){
111         *head = currNode->next;
112     } else {
113         prevNode->next = currNode->next;
114     }
115
116     delete currNode;
117 }
118
119 template <typename T>
120 void sllDeleteList(SinglyList<T>** head){
121
122     SinglyList<T>* current = *head;
123
124     while(current != nullptr){
125         SinglyList<T>* temp = current;
126         current = current->next;
127         delete temp;
128     }
129
130     *head = nullptr;
131 }
132
133 #endif
```

```cpp
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H

//Doublelist Node
template <typename T>
class DoubleList{
    public:
        T data;
        DoubleList<T>* next;
        DoubleList<T>* prev;
};

//Creating a node function
template<typename T>
DoubleList<T>  *createNode(T newData){
    //Dynamic memory alloction for a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    //Point it to the next node
    newNode->prev = nullptr;
    newNode->next = nullptr;

    return newNode;
}

template<typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //Check if the list is empty
    if(currentNode == nullptr){
        std::cout << "The list is empty";
        return;
    }

    DoubleList<T>* tail;
    std::cout << "FORWARD" << std::endl;
    //Forward traverse
    while(currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode  = currentNode->next;
    }
    std::cout << std::endl;
    //Reverse traverse
```

```
45      std::cout << "REVERSE" << std::endl;
46      while(tail != nullptr){
47          std::cout << tail->data << " ";
48          tail = tail->prev;
49      }
50      std::cout << std::endl;
51
52  }
53
54  //inserting a node to the head
55  template <typename T>
56  void dllInsertHead(T newData, DoubleList<T>** currentNode){
57
58      // create a new node:
59      DoubleList<T> *newNode = createNode(newData);
60      //point the newNode to the currentNode (head)
61      newNode->next = *currentNode;
62      //point the currentNode(head) to the newNode
63      (*currentNode)->prev = newNode;
64
65  }
66
67  //inserting at the End;
68  template <typename T>
69  void dllInsertEnd(T newData, DoubleList<T> *currentHead){
70      //create a new node
71      DoubleList<T>* newNode = createNode(newData);
72
73      //traverse until we reach the last node
74      while(currentHead->next != nullptr){
75          currentHead = currentHead->next;
76      }
77
78      //connect the new node back to the last node
79      newNode->prev = currentHead;
80      //connect the curent node to the new node
81      currentHead->next = newNode;
82  }
83  template<typename T>
84  void dllGeneralInsert(T newData, DoubleList<T>* preNode){
85      if(preNode == nullptr){
86          std::cout << "Previous value cannot be null" << std::endl;
```

```
87      }
88      DoubleList<T>* newNode = createNode(newData);
89
90      newNode->next = preNode->next;
91      newNode->prev = preNode;
92      preNode->next = newNode;
93      (preNode->next->next)->prev = newNode;
94  }
95
96  //deleting a node
97  template<typename T>
98  void dllDeleteNode(T findData, DoubleList<T>** head){
99      if(*head == nullptr)return;
00
01      DoubleList<T>* currNode = *head;
02      DoubleList<T>* preNode = nullptr;
03
04      while(currNode != nullptr && currNode->data !=findData){
05          preNode = currNode;
06          currNode = currNode->next;
07      }
08      if(currNode == nullptr){
09          std::cout << "Data is not found." << std::endl;
10          return;
11      }
12      if(preNode == nullptr){
13          *head = currNode->next;
14          (*head)->prev = nullptr;
15      }else{
16          preNode->next = currNode->next;
17          (currNode->next)->prev = preNode;
18      }
19      delete currNode;
20  }
21
22  #endif
23
24
```

```
FORWARD
C P E
REVERSE
E P C
Testing the dllInsertHead
FORWARD
C P E
REVERSE
E P C X
Testing the dllInsertENd
FORWARD
C P E 0
REVERSE
0 E P C X

Testing General Insert
FORWARD
C P E 1 0
REVERSE
0 1 E P C X

Testing delete a node
Data is not found.
FORWARD
REVERSE
0 1 E P C X

Testing delete a node
Data is not found.
REVERSE
0 1 E P C X

Testing delete a node
REVERSE
0 1 E P C X

REVERSE
REVERSE
0 1 E P C X
REVERSE
0 1 E P C X
REVERSE
0 1 E P C X

Testing delete a node
REVERSE
0 1 E P C X
```

```
Testing delete a node
Data is not found.
FORWARD
REVERSE
0 1 E P C X

Testing delete a node
REVERSE
0 1 E P C X

Testing delete a node
Data is not found.
REVERSE
0 1 E P C X

REVERSE
0 1 E P C X

REVERSE
0 1 E P C X
REVERSE
0 1 E P C X
REVERSE
REVERSE
REVERSE
0 1 E P C X
0 1 E P C X

Testing delete a node
Data is not found.
FORWARD
0 1 E P C X

Testing delete a node
Data is not found.
0 1 E P C X

Testing delete a node
0 1 E P C X

Testing delete a node
0 1 E P C X


Testing delete a node
Data is not found.
FORWARD
Data is not found.
FORWARD
FORWARD
C P E 1 0
REVERSE
0 1 E P C X
PS C:\Users\TIPQC\Documents\joel a almirol>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.